FACULDADE DE CIÊNCIAS E TECNOLOGIA DA UNIVERSIDADE DE COIMBRA

DEPARTAMENTO DE ENGENHARIA INFORMÁTICA



Fundamentos de Inteligência Artificial

Trabalho 2

Trabalho realizado por:

Cláudio Catarino Nº2022224320 uc2022224320@student.uc.pt

Inês Nisa Nº2022247346 uc2022247346@student.uc.pt

Miguel Calado Nº2022228235 uc2022228235@student.uc.pt

PL6

Índice

Introdução.....3

Funcionalidades Implementadas 4

Experiências Realizadas e Resultados Obtidos 5

Análise De Resultados.....7

Alterações feitas com a presença de vento 9

Conclusão 10

Introdução

No âmbito da cadeira de Fundamentos de Inteligência Artificial foi-nos proposto o desenvolvimento de um projeto com o objetivo de adquirir competências de desenho e implementação de Algoritmos Evolucionários para a criação de agentes autónomos. Para isso utilizámos o ambiente Lunar Lander onde o agente deve ser capaz de aterrar a nave na plataforma

Neste projeto, esses algoritmos foram aplicados à evolução de redes neuronais que controlam o comportamento do agente, dessa forma o agente é treinado através de sucessivas simulações no ambiente, sendo avaliado com base no seu desempenho, o que permite a melhoria contínua do seu comportamento.

Funcionalidades Implementadas

Neste trabalho foi-nos dado um código em python para nós completarmos de maneira que o agente baseado em neuroevolução fosse capaz de aterrar com sucesso.

No código fornecido, destacavam-se 4 funções que tínhamos de desenvolver ou melhorar de forma a aumentar o desempenho do agente na aterragem pretendida:

- **def objective_function (),** esta função é responsável por calcular a aptidão do agente de forma a evoluir o comportamento desejado. Nesta função começamos por ir buscar distância horizontal ao centro da zona de aterragem(x), altura da nave em relação ao solo(y), velocidade horizontal(vx), velocidade horizontal(vy), ângulo de inclinação da nave(theta), velocidade de rotação(omega). Estas observações vão servir para calcular as penalidades, que representam o quão longe o agente se encontra de um estado ideal de aterragem.
  São as seguintes penalidades:

  - ***pos_penalty***, onde sumamos os valores absolutos da posição horizontal e vertical, para penalizar por estar demasiado longe de onde a nave tem que aterrar e por estar demasiado alto.
  - ***vel_penalty***, onde sumamos os valores absolutos da velocidade horizontal e vertical, para penalizar movimentos demasiado rápidos.
  - ***angle_penalty***, valor absoluto da inclinação da nave, para penalizar aterragens com grande inclinação.
  - ***rotation_penalty***, valor absoluto da velocidade angular para penalizar quando ocorre demasiadas rotações durante a aterragem.

  De seguidas já com estas penalidades feitas, calculamos o fitness, sendo a soma negativa de todas as penalidades, tendo mais impacto as pos_penalty e a vel_penalty de seguida com metade do impacto a angle_penalty e de seguida só com 0.2 a rotation_penalty. No final se a nave aterrou corretamente é lhe acrescentado 200 pontos de aptidão.

Esta função favorece os agentes que aterram de forma suave, com precisão e com estabilidade, fazendo com que o processo de evolução tenha comportamentos cada vez mais seguros.

- **def mutation (p, generation),** esta função é responsável por introduzir variação nos indivíduos ao longo das gerações, ele aplica mutações controladas ao genótipo alterando ligeiramente os seus genes. Esta função começa por calcular o annealed_std que corresponde ao desvio padrão da mutação que vai diminuindo ao longo das gerações, depois para cada gene decide se vai ser mutado e caso seja, é aplicado perturbação aleatória ao gene para manter a variedade.

- **def parent_selection(population),** esta função é responsável por selecionar um individuo da população para ser progenitor sendo que quanto melhor for o individuo maior probabilidade tem de ser escolhido. Esta função escolhe 10 indivíduos da população e ordena-os pela sua aptidão, de seguida vai buscar o que tem a melhor aptidão e retorna uma copia do mesmo. Isto garante que se passem os melhores genes.

- ***def crossover (p1, p2),*** esta função é responsável por criar um novo indivíduo combinando genes de dois progenitores de maneira a retirar informação vantajosa de ambos, uma vez que p1 e p2 foram definidos pela **parent_selection** onde dentro do grupo de indivíduos eles foram os melhores. Esta função o que faz é itera sobre p1 e p2, define beta que vai ser um número aleatório entre (-alpha, 1 + alpha) e depois combina os genes para criar um novo indivíduo com uma mistura dos genes dos pais. Com este tipo de algoritmo os filhos conseguem ficar fora do intervalo definido pelos pais, alcançando uma maior exploração espaço de soluções.

Experiências Realizadas e Resultados Obtidos

Para avaliarmos o desempenho do algoritmo evolutivo implementado tivemos de realizar 8 experiências com valores de parâmetros diferentes listados no enunciado do projeto. O valor da população e das gerações foi sempre o mesmo o que mudou principalmente foram os valores mutação, crossover e etilismo, isto para posteriormente se observar como cada configuração influenciou o desempenho do algoritmo evolutivo.

<table>
<tr><th></th><th>Mutação</th><th>Crossover</th><th>Elitismo</th><th>Tamanho da População</th><th>Gerações</th></tr>
<tr><td>Experiência 1</td><td>0.008</td><td rowspan="2">0.5</td><td rowspan="4">0</td><td rowspan="8">100</td><td rowspan="8">100</td></tr>
<tr><td>Experiência 2</td><td>0.05</td></tr>
<tr><td>Experiência 3</td><td>0.008</td><td rowspan="2">0.9</td></tr>
<tr><td>Experiência 4</td><td>0.05</td></tr>
<tr><td>Experiência 5</td><td>0.008</td><td rowspan="2">0.5</td><td rowspan="4">1</td></tr>
<tr><td>Experiência 6</td><td>0.05</td></tr>
<tr><td>Experiência 7</td><td>0.008</td><td rowspan="2">0.9</td></tr>
<tr><td>Experiência 8</td><td>0.05</td></tr>
</table>

De forma a obtermos resultados mais fiáveis, realizámos cada uma das experiências indicadas acima 5 vezes, fazendo depois a média de acertos nessas 5 repetições. Em cada repetição de uma determinada experiência fizemos 1000 testes. Nas tabelas abaixo, estão indicadas as médias de acerto, em percentagem, anteriormente referidas.

Sem vento

Experiência	Mutação	Crossover	Elitismo	População	Gerações	% Acertos
1	0.008	0.5	0	100	100	83.44%
2	0.05	0.5	0	100	100	89.50%
3	0.008	0.9	0	100	100	81.70%
4	0.05	0.9	0	100	100	95.34%
5	0.008	0.5	1	100	100	87.40%
6	0.05	0.5	1	100	100	91.76%
7	0.008	0.9	1	100	100	78.48%
8	0.05	0.9	1	100	100	90.94%

Com vento

Experiência	Mutação	Crossover	Elitismo	População	Gerações	% Acertos
1	0.008	0.5	0	100	100	54.06%
2	0.05	0.5	0	100	100	83.44%
3	0.008	0.9	0	100	100	60.28%
4	0.05	0.9	0	100	100	76.07%
5	0.008	0.5	1	100	100	73.22%
6	0.05	0.5	1	100	100	76.43%
7	0.008	0.9	1	100	100	74.34%
8	0.05	0.9	1	100	100	79.53%

Análise De Resultados

Sem vento:

Após analisarmos os resultados foi-nos possível observar que as experiências que têm uma maior probabilidade de mutação (0.05), as experiências 2, 4, 6 e 8 têm uma percentagem maior de acertos em comparação com as restantes que têm uma probabilidade de mutação mais baixa (0.008), isto pode ser devido ao facto de a mutação introduzir uma maior diversidade da população. Um fator de mutação mais alto permite o acréscimo de novas características aos indivíduos, fazendo com que ao longo das gerações os indivíduos venham a ter mais probabilidade de ter desempenhos melhores.

Quando o crossover está baixo (0.5) ou alto (0.9) e a probabilidade de mutação está baixa (0.008) tivemos resultados semelhantes, uma vez que como a chance de algum gene mudar é baixa, mesmo havendo crossover vai haver apenas uma mistura de genes parecidos. Por outro lado, se o crossover estiver alto (0.9) e a probabilidade de mutação também (0.05) há uma melhora significativa nos acertos pois, vamos ter uma mistura de genes que sofreram uma probabilidade de mutação maior, ou seja, vamos estar a fazer uma mistura com genes mais variados.

Em relação ao elitismo não houve muitas diferenças com e sem. Isso pode estar relacionado com o facto de estarmos a usar uma amostra pequena para a elite. No entanto, quando existe elitismo vamos ter uma percentagem de acertos mais estável, ou seja, com uma diferença menor entre as experiências realizadas, em comparação com as experiências sem elitismo.

Com vento:

Com a presença do vento verificou-se que a mutação continuou a dar melhores resultados quando era mais elevada, isto significa que mesmo com fatores externos a variedade de genes continua a ter um bom desempenho.

Em relação ao elitismo, com a presença de vento e estando o elitismo a 1 verificou-se uma estabilidade nos resultados da percentagem dos acertos, o que não aconteceu quando o elitismo estava a 0. Isto indica que mesmo com o fator externo do vento e elitismo contribui para melhores resultados.

Para o crossover baixo e alto quando combinado com uma probabilidade de mutação baixa (0.008) tivemos resultados semelhantes, uma vez que como a chance de algum gene mudar é baixa, mesmo havendo crossover vai haver apenas uma mistura de genes parecidos. Por outro lado, tal como nas experiências sem vento, se o crossover estiver alto (0.9) e a probabilidade de mutação também (0.05) há uma melhora significativa nos acertos pois há maior diversidade pois temos uma mistura entre genes que diferenciam mais entre si.

Concluindo

Sem vento:

- Mutação elevada melhorou o desempenho
- Crossover elevado (0.9) aumenta resultados apenas quando também existe uma probabilidade de mutação mais alta (0.05)
- Elitismo não houve muitas diferenças significativas
- A experiência com os melhores resultados foi a experiência 4 com probabilidade de mutação de 0.05 e crossover 0.9 e sem elitismo
- Elitismo permite resultados mais estáveis

Com vento:

- Mutação elevada continua a melhorar o desempenho
- Crossover elevado (0.9) aumenta resultados apenas quando também existe uma probabilidade de mutação mais alta (0.05)
- Com Elitismo verificou-se resultados mais estáveis
- A experiência com os melhores resultados foi a experiência 2 com probabilidade de mutação de 0.05 e crossover 0.5 e sem elitismo

Alterações feitas com a presença de vento

Para a meta 2 o objetivo era fazer a mesma coisa que para a meta 1 mas com a presença do vento. Para tal primeiramente tivemos de colocar o ENABLE_WIND a True para ativar o vento no ambiente. Para o agente aterrar melhor com o vento a alterações que fixemos foi no **def objective_function (),** onde colocamos um if ENABLE_WIND para caso o vento esteja ativo fazer aquilo. Dentro do if, a principal diferença que fizemos para o agente lidar com o vento foi tornar as penalizações menos severas, isto porque o vento vai ser um fator externo que o agente não controla por isso não podemos penalizar tao vigorosamente por desvios que não refletem diretamente no agente, já se for sem a

presença de vento a culpa cai toda sobre o agente e por isso tem de ser mais penalizado para corrigir o erro que cometeu.

Verificamos que com vento a média de acerto nas aterragens é menor, o que é normal devido á maior dificuldade provocada pelo mesmo.

Conclusão

Em suma este projeto permitiu aplicar algoritmos evolucionários para a criação de agentes autónomos capazes de controlar a aterragem de uma nave no ambiente Lunar Lander. Através da evolução de redes neuronais, foi possível desenvolver comportamentos adaptativos sem intervenção humana direta.